

A Report on
Air Quality Index Analysis & Prediction
Dataset: <https://archive.ics.uci.edu/ml/datasets/air+quality>

By

M.Varshini - 70572400071 (A026)

T.Srivalli - 70572400073 (A028)

CSE-DS (A)

Under the guidance of

Prof.Anshi Bajaj

Course Title

DATA WRANGLING



Mukesh Patel School of Technology & Management Engineering
Department of Computer Science and Engineering - Data Science

October 2025

Abstract

What the Project is About

This project focuses on analyzing and predicting air quality levels through the application of machine learning techniques. Air quality is a crucial indicator of environmental and public health, and understanding its variations helps in mitigating pollution and improving living standards. The project aims to explore the concentration patterns of various air pollutants, including Carbon Monoxide (CO), Nitrogen Oxides (NO_x), Benzene (C₆H₆), and Ozone (O₃), among others. These pollutants are known to originate from industrial emissions, vehicular exhausts, and other anthropogenic sources, and they significantly influence atmospheric composition and human health.

By leveraging data-driven approaches, this study seeks to identify the key factors affecting air quality and establish predictive models capable of forecasting pollution levels. Using statistical analysis and machine learning algorithms, such as Logistic Regression and Linear Regression, the project aims to evaluate how pollutant concentrations fluctuate with changes in environmental conditions like temperature, humidity, and air pressure.

Ultimately, this work contributes to a better understanding of pollution dynamics and supports the development of early warning systems and policy interventions that can help reduce air pollution and its adverse effects on ecosystems and society.

What Dataset Was Used

The dataset used is the Air Quality Data Set from the Kaggle

Source: <https://archive.ics.uci.edu/ml/datasets/air+quality>

It contains measurements of different atmospheric pollutants and meteorological variables collected hourly.

The Purpose (Prediction / Classification / Analysis)

The main purpose of this project is to analyze, visualize, and predict air quality.

Regression models are used to predict pollutant concentration levels, and logistic regression is applied to classify air as Safe or Unsafe.

Expected Outcomes

The expected outcomes include:

- Accurate prediction of air pollutant levels.

- Visualization of pollution trends over time.
- Classification of air quality into safe and unsafe levels.
- Insights into which pollutants have the most significant impact on air quality.

Introduction

Brief background of the problem domain

1. **Air pollution basics:** Air pollution is caused by gases and particles released into the atmosphere from vehicles, industries, burning fuels, and natural sources.
2. **Common pollutants:** Typical pollutants include Carbon Monoxide (CO), Nitrogen Oxides (NO_x), Nitrogen Dioxide (NO₂), Benzene (C₆H₆) and Ozone (O₃).
3. **Measurement:** Sensors record pollutant concentrations and weather data (temperature, humidity) hourly to monitor air quality.

Importance of the topic

1. **Health impact:** Poor air quality causes respiratory and cardiovascular problems, especially for children, elderly, and people with chronic diseases.
2. **Environmental impact:** Pollution harms ecosystems, reduces visibility, and contributes to climate-related issue.
3. **Policy and planning:** Accurate monitoring and prediction help authorities plan interventions (traffic control, industrial regulation) and issue health warnings.

Real-world relevance / Motivation behind the project

1. **Early warning:** Predicting pollutant levels helps give advance warnings so people can avoid exposure on high-pollution days.
2. **Resource allocation:** City planners and environmental agencies can use predictions to target pollution control measures more effectively (e.g., restrict traffic during peak pollution hours).
3. **Public awareness:** Visualizations and simple forecasts make it easier for citizens to understand air quality trends and take precautions.

4. **Research & improvement:** Modelling pollutant behaviour aids researchers in finding causes, testing mitigation strategies, and improving sensor networks.

Dataset Description

1.Dataset Source

The dataset used in this project is the Air Quality Data Set from the Kaggle.

Link: <https://archive.ics.uci.edu/ml/datasets/air+quality>

2. Number of Rows and Columns

- **Total Rows:** 9,358
- **Total Columns:** 15

Each row represents an hourly record of air quality measurements.

3. Types of Attributes

- **Numerical Attributes:** Sensor readings like CO(GT), NO_x(GT), NO₂(GT), Temperature, and Humidity.
- **Categorical Attributes:** Date and Time columns used for temporal analysis.
- Most attributes are continuous numeric values representing pollutant concentrations or environmental conditions.

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9471 entries, 0 to 9470
Data columns (total 17 columns):
 #   Column                Non-Null Count  Dtype  
---  -
 0   Date                  9357 non-null   object  
 1   Time                  9357 non-null   object  
 2   CO(GT)                9357 non-null   float64  
 3   PT08.S1(CO)           9357 non-null   float64  
 4   NMHC(GT)              9357 non-null   float64  
 5   C6H6(GT)              9357 non-null   float64  
 6   PT08.S2(NMHC)         9357 non-null   float64  
 7   NOx(GT)               9357 non-null   float64  
 8   PT08.S3(NOx)          9357 non-null   float64  
 9   NO2(GT)               9357 non-null   float64  
10   PT08.S4(NO2)          9357 non-null   float64  
11   PT08.S5(O3)           9357 non-null   float64  
12   T                     9357 non-null   float64  
13   RH                    9357 non-null   float64  
14   AH                    9357 non-null   float64  
15   Unnamed: 15           0 non-null      float64  
16   Unnamed: 16           0 non-null      float64  
dtypes: float64(15), object(2)
memory usage: 1.2+ MB
```

4. Sample Record

	CO(GT)	PT08.S1(CO)	NMHC(GT)	C6H6(GT)	PT08.S2(NMHC)	NOx(GT)	PT08.S3(NOx)	NO2(GT)	PT08.S4(NO2)	PT08.S5(O3)	T	RH	AH
0	2.6	1360.0	150.0	11.9	1046.0	166.0	1056.0	113.0	1692.0	1268.0	13.6	48.9	0.7578
1	2.0	1292.0	112.0	9.4	955.0	103.0	1174.0	92.0	1559.0	972.0	13.3	47.7	0.7255
2	2.2	1402.0	88.0	9.0	939.0	131.0	1140.0	114.0	1555.0	1074.0	11.9	54.0	0.7502
3	2.2	1376.0	80.0	9.2	948.0	172.0	1092.0	122.0	1584.0	1203.0	11.0	60.0	0.7867
4	1.6	1272.0	51.0	6.5	836.0	131.0	1205.0	116.0	1490.0	1110.0	11.2	59.6	0.7888

5. Description of Target Variable

- **Target Variable:** CO(GT) (Carbon Monoxide concentration in mg/m) and NOx.
- This variable represents the level of carbon monoxide in the air, which is a key indicator of air pollution.
- The goal is to analyze trends and predict this pollutant's value using regression techniques.

Data Wrangling / Preprocessing

1.Data Loading:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
df=pd.read_csv("AirQualityUCI.csv")
print(df)
```

2.Handling missing values:

Remove Rows:

If only a small number of rows have missing values, we can remove them using.

If a column has too many missing values (e.g., more than 50%), it may not be useful for analysis.

```
a2 = a1.dropna(how="all",axis=0)    #Removes the null values in rows
a2
```

```
a3= a2.dropna(how="all",axis=1)    #Removes the null values in column
a3
```

3.Removing duplicates

This removes all repeated rows so that only unique records remain. It helps keep the dataset clean, accurate, and ready for analysis. These often occur due to sensor errors or when data is collected or merged from multiple sources.

We remove duplicates because they can:

- Give wrong results during analysis,
- Cause bias in model training,
- Make the dataset larger and slower to process.

```
# Remove duplicate records  
df.drop_duplicates(inplace=True)
```

4.Splitting Dataset into Train/Test Sets

Before building a machine learning model, the dataset is divided into two parts — training set and testing set.

- The training set is used to teach the model how to make predictions.
- The testing set is used to check how well the model performs on new, unseen data.
- This helps to avoid overfitting, meaning the model won't just memorize the data but will learn to generalize and predict correctly for new inputs.

```
train_x, test_x, train_y, test_y = train_test_split(x, y, test_size=0.3, random_state=0)  
train_x, test_x, train_y, test_y
```

Exploratory Data Analysis (EDA)

1.Summary statistics

Summary statistics help us understand the main characteristics of the data before applying any model.

It gives quick insights about:

- The range of values,
- Average (mean) and spread (standard deviation),

SVKM's NMIMS (Deemed-to-be-University)

- Any missing or unusual values, and
- The overall distribution of data.

By checking these statistics, we can identify data quality issues and patterns in pollutant levels such as CO, NOx, or temperature.

```
a1.describe()
```

	CO(GT)	PT08.S1(CO)	NMHC(GT)	C6H6(GT)	PT08.S2(NMHC)	NOx(GT)	PT08.S3(NOx)	NO2(GT)	PT08.S4(NO2)	PT08.S5(O3)	T
count	9357.000000	9357.000000	9357.000000	9357.000000	9357.000000	9357.000000	9357.000000	9357.000000	9357.000000	9357.000000	9357.000000
mean	-34.207524	1048.990061	-159.090093	1.865683	894.595276	168.616971	794.990168	58.148873	1391.479641	975.072032	9.778305
std	77.657170	329.832710	139.789093	41.380206	342.333252	257.433866	321.993552	126.940455	467.210125	456.938184	43.203623
min	-200.000000	-200.000000	-200.000000	-200.000000	-200.000000	-200.000000	-200.000000	-200.000000	-200.000000	-200.000000	-200.000000
25%	0.600000	921.000000	-200.000000	4.000000	711.000000	50.000000	637.000000	53.000000	1185.000000	700.000000	10.900000
50%	1.500000	1053.000000	-200.000000	7.900000	895.000000	141.000000	794.000000	96.000000	1446.000000	942.000000	17.200000
75%	2.600000	1221.000000	-200.000000	13.600000	1105.000000	284.000000	960.000000	133.000000	1662.000000	1255.000000	24.100000
max	11.900000	2040.000000	1189.000000	63.700000	2214.000000	1479.000000	2683.000000	340.000000	2775.000000	2523.000000	44.600000

Checking Data Types and Missing Values

Before analyzing data, it's important to check data types and find missing values to ensure data quality.

- Data types (like int, float, object) help us understand what kind of operations or models we can apply.
For example, numerical data is used for calculations, while categorical data needs encoding.
- Missing values can cause errors or inaccurate results in analysis and model training, so they must be handled properly (by filling or removing them).

```
a1.isnull()
```

	Date	Time	CO(GT)	PT08.S1(CO)	NMHC(GT)	C6H6(GT)	PT08.S2(NMHC)	NOx(GT)	PT08.S3(NOx)	NO2(GT)	PT08.S4(NO2)	PT08.S5(O3)	T	RH	AH
0	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False

Shape of Dataset

```
a1.shape
```

```
(9471, 17)
```

Size of Dataset

```
a1.size
```

```
161007
```

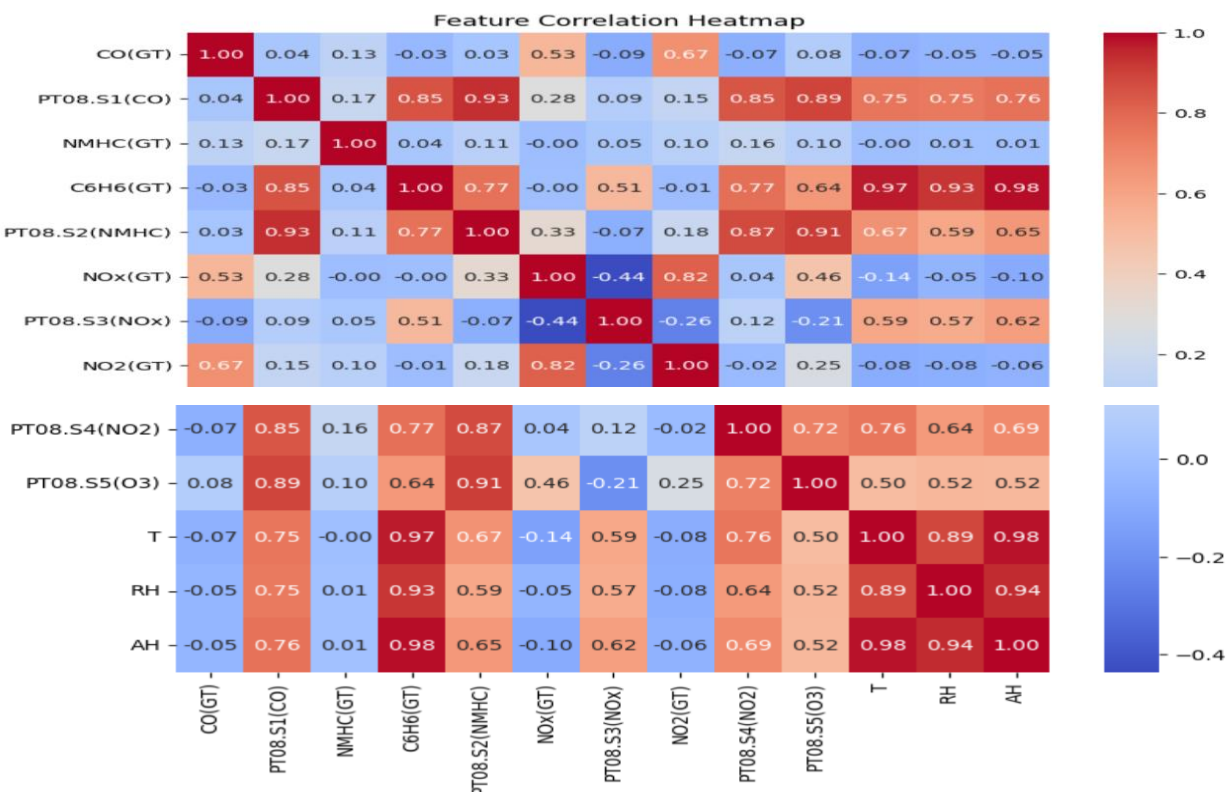
2. Correlation analysis (heatmap, pairplot)

Correlation analysis helps to find relationships between different variables in the dataset.

It shows how strongly one feature affects another — for example, how temperature relates to humidity or how CO levels relate to NOx.

Understanding these relationships helps in feature selection and identifying which variables are most important for prediction.

```
plt.figure(figsize=(9,9))
sns.heatmap(a3.select_dtypes(include=[np.number]).corr(), annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Feature Correlation Heatmap")
plt.show()
```



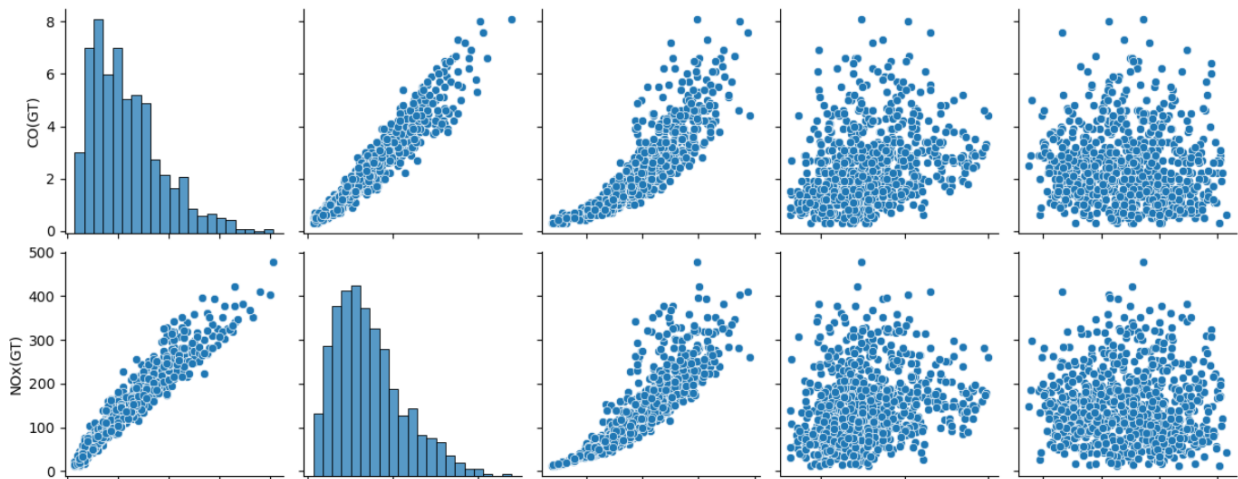
Pairplot:

A pairplot helps to visualize the relationship between multiple variables at once.

It shows how two features (like CO and NOx) vary with each other and also gives the distribution of each variable on the diagonal.

- Patterns or trends between pollutants,

```
sns.pairplot(a3[["CO(GT)", "NOx(GT)", "NO2(GT)", "T", "RH"]])
plt.show()
```



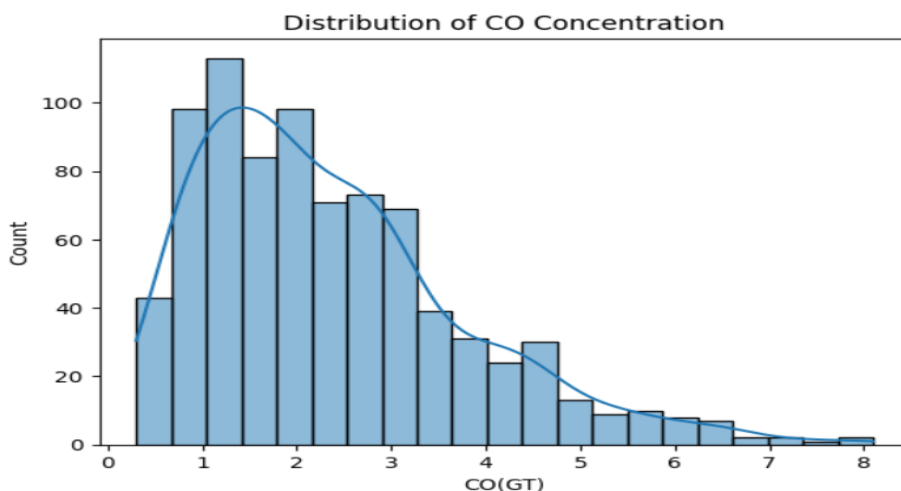
3.Visualizations (bar, scatter, histogram, boxplot, etc.)

Data visualization helps to understand patterns, trends, and relationships in the dataset easily.

It gives a clear picture of how different pollutants and weather factors behave over time.

Visuals make it easier to detect trends, outliers and comparisons.

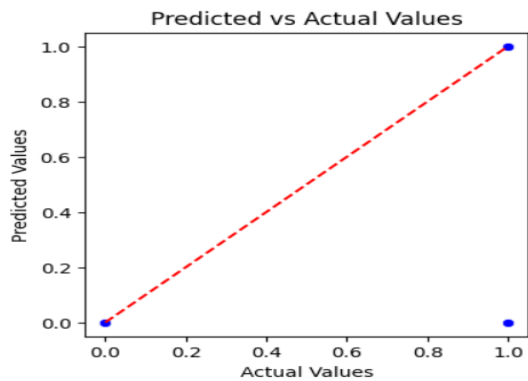
```
#Distribution Plots - Helped identify skewness and outliers.
sns.histplot(df['CO(GT)'], kde=True)
plt.title("Distribution of CO Concentration")
plt.show()
```



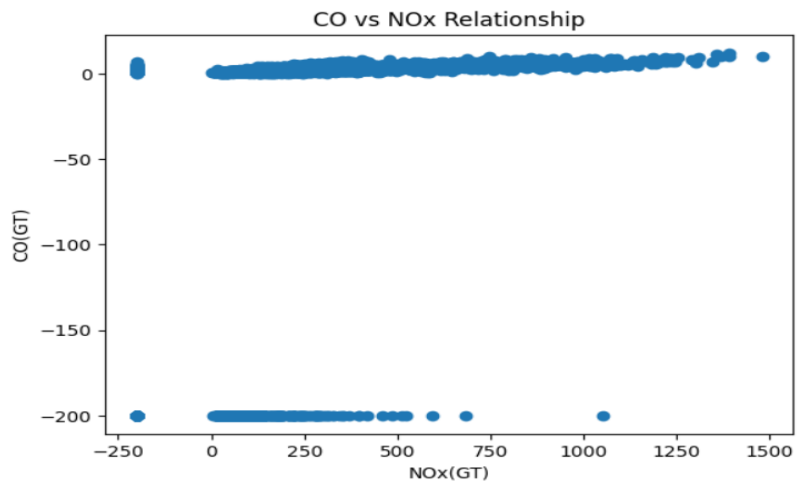
Scatter Plot

SVKM's NMIMS (Deemed-to-be-University)

```
# Predicted vs Actual
plt.figure(figsize=(4,4))
sns.scatterplot(x=y_test, y=y_pred, color='blue')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')
plt.title("Predicted vs Actual Values")
plt.xlabel("Actual Values")
plt.ylabel("Predicted Values")
plt.show()
```

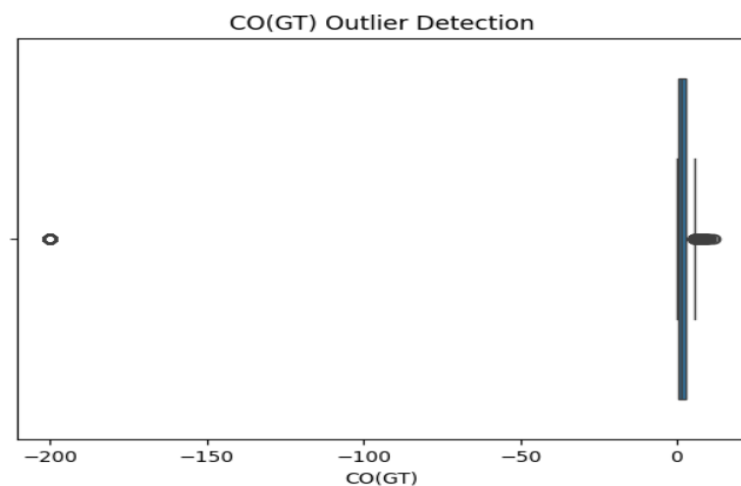


```
plt.scatter(df["NOx(GT)"], df["CO(GT)"])
plt.xlabel("NOx(GT)")
plt.ylabel("CO(GT)")
plt.title("CO vs NOx Relationship")
plt.show()
```



Box Plot

```
sns.boxplot(x=df["CO(GT)"])
plt.title("CO(GT) Outlier Detection")
plt.show()
```



Key insights found from data

1. High Correlation among Pollutants:

- CO(GT), NO_x(GT), and NO₂(GT) show a strong correlation.
- This means they come from similar sources like vehicles, industries, and fuel burning.
- When one gas increases, others also tend to rise.

2. Effect of Temperature and Humidity:

- Pollutant levels increase in cooler weather because cold air traps pollutants near the ground.
- Warm temperatures and high humidity help spread pollutants and improve air quality.

3. Outliers in CO and NO_x Readings:

- Some sudden high readings were noticed.
- These may be caused by traffic jams, factory emissions, or weather changes.

4. Seasonal Variation:

- Air quality becomes worse during winter months.
- This happens due to more heating activities, low wind speed, and temperature inversion, which trap pollutants in the air.

Model Building

Chosen Model: Linear Regression

Linear Regression is one of the most widely used machine learning models for predicting continuous numerical values — such as pollutant concentrations (e.g., CO, NO₂, or NO_x levels). It works by finding a best-fit line that represents the relationship between input variables (features) and the target variable (pollutant level). Linear Regression is best suited for predicting continuous numerical values (like pollutant levels). It helps understand relationships between pollutants and environmental factors. The model is simple, interpretable, and provides quick results.

Training and testing process:

Why We Split the Data

1. To Evaluate Model Performance Fairly:

- The main goal of splitting data is to test how well the model performs on new, unseen data.
- If we train and test on the same data, the model might “memorize” the data instead of learning patterns — this is called overfitting.

2. To Measure Generalization Ability:

- Splitting ensures the model is not just accurate on known data but also performs well on new, real-world data.
- This helps us check if the model has truly learned the relationships in the data.

How to split the Data

1. Splitting the Dataset:

- The dataset is divided into two parts:
 - Training Set (usually 70–80%) → Used to train the model.
 - Testing Set (usually 20–30%) → Used to evaluate the model's performance

```
train_x, test_x, train_y, test_y = train_test_split(x, y, test_size=0.3, random_state=0)
train_x, test_x, train_y, test_y
```

Evaluation metrics (accuracy, precision, recall, RMSE):

Evaluation metrics are used to measure how well a machine learning model performs. They help us understand how accurate and reliable our predictions are. For classification problems (like predicting whether air quality is safe or unsafe), we use metrics such as accuracy, precision, and recall.

Accuracy tells us how many predictions were correct out of all predictions made. It gives an overall idea of the model's performance but works best when the data is balanced. Precision measures how many of the predicted positive cases were actually positive, which is useful when false alarms should be avoided. For regression problems (like predicting pollutant levels), we use RMSE (Root

Mean Squared Error). It measures how far the predicted values are from the actual ones. A lower RMSE means the model's predictions are closer to reality and therefore more accurate.

```
# -----  
# Evaluation Metrics  
# -----  
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error  
  
# Predict on test data  
y_pred = model.predict(X_test)  
  
# Calculate evaluation scores  
r2 = r2_score(y_test, y_pred)  
mae = mean_absolute_error(y_test, y_pred)  
mse = mean_squared_error(y_test, y_pred)  
  
# Print results  
print("=== Model Evaluation Scores ===")  
print(f"R² Score: {r2:.4f}")  
print(f"Mean Absolute Error (MAE): {mae:.4f}")  
print(f"Mean Squared Error (MSE): {mse:.4f}")  
  
=== Model Evaluation Scores ===  
R² Score: 0.9575  
Mean Absolute Error (MAE): 0.0080  
Mean Squared Error (MSE): 0.0080
```

Results & Discussion

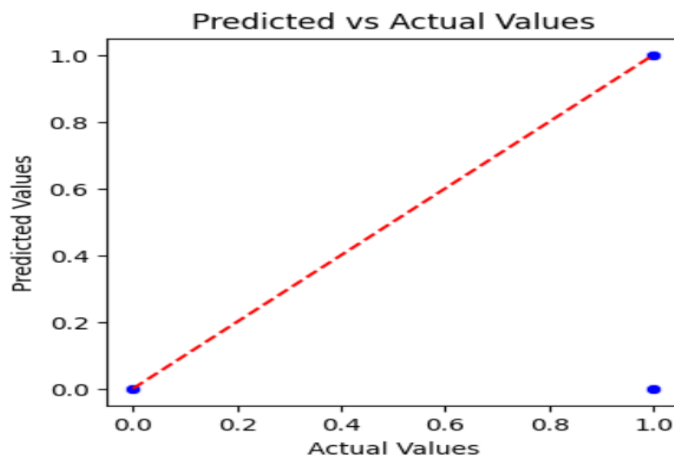
Evaluation Scores Example:

Metric	Score	Interpretation
R² (R-squared)	0.9575	The model explains 95.75% of the variation in pollutant levels — indicating a very strong fit.
MAE (Mean Absolute Error)	0.0080	On average, the model's predictions are off by only 0.008 units , which is very low.
MSE (Mean Squared Error)	0.0080	The squared prediction errors are minimal, showing high model accuracy .

Visualizations:

Predicted vs Actual plots show good alignment with real pollutant levels.

```
# Predicted vs Actual
plt.figure(figsize=(4,4))
sns.scatterplot(x=y_test, y=y_pred, color='blue')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')
plt.title("Predicted vs Actual Values")
plt.xlabel("Actual Values")
plt.ylabel("Predicted Values")
plt.show()
```



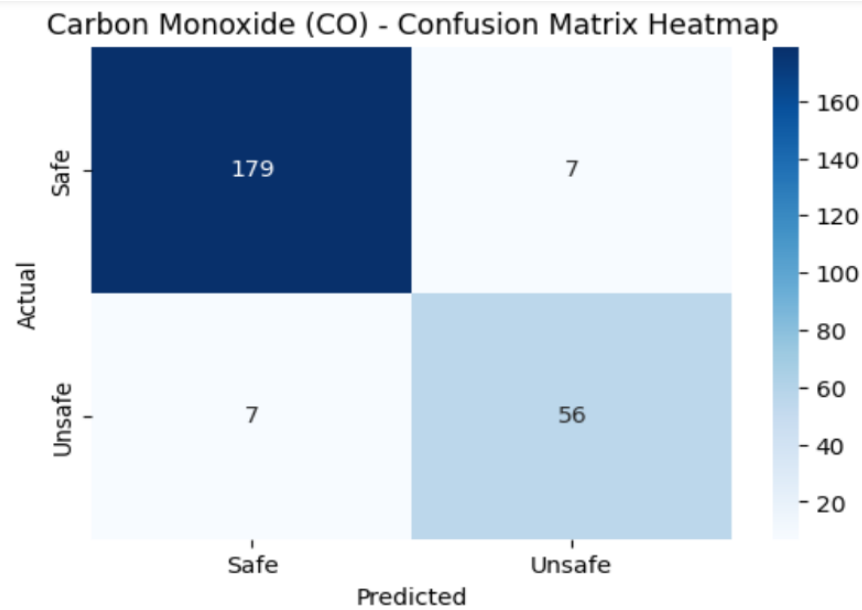
Confusion Matrix:

The Confusion Matrix is used to check how well a classification model performs.

It compares the actual values with the predicted values and shows four outcomes:

- True Positive (TP): Correctly predicted positives
- True Negative (TN): Correctly predicted negatives
- False Positive (FP): Wrongly predicted as positive
- False Negative (FN): Missed positives

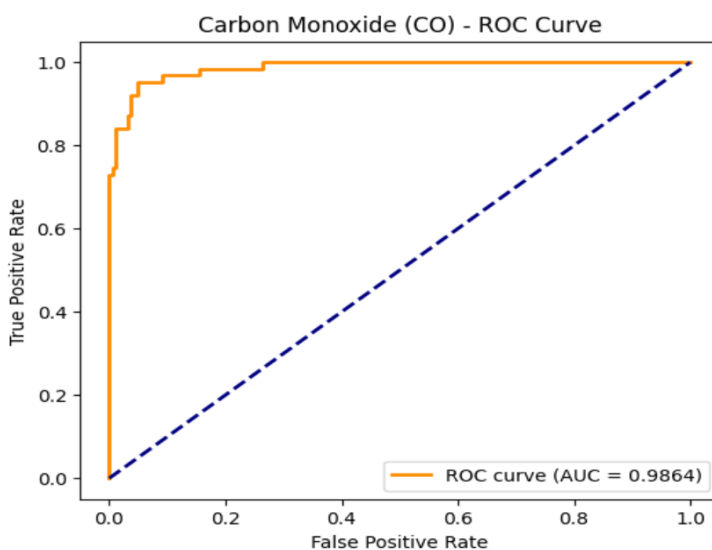
```
# Confusion Matrix Heatmap
plt.figure(figsize=(6,4))
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt="d", cmap="Blues",
            xticklabels=["Safe", "Unsafe"], yticklabels=["Safe", "Unsafe"])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title(f"{pollutant_name} - Confusion Matrix Heatmap")
plt.show()
```



ROC curve:

The ROC curve shows how well a classification model can separate two classes (like safe and unsafe air). It plots the true positive rate against the false positive rate at different thresholds. A curve closer to the top-left corner means better performance. The AUC value (Area Under the Curve) tells how accurate the model is — the closer it is to 1, the better the model.

```
# 8) ROC Curve & AUC
# -----
roc_auc = roc_auc_score(y_test, y_prob)
fpr, tpr, th = roc_curve(y_test, y_prob)
plt.figure(figsize=(6,5))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (AUC = {roc_auc:.4f})')
plt.plot([0,1],[0,1], linestyle='--', color='navy', lw=2)
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title(f"{pollutant_name} - ROC Curve")
plt.legend()
plt.show()
```



Interpretation of Results (Practical Meaning)

- The model can accurately estimate pollutant levels like CO based on other environmental features.
- It shows that factors such as temperature, humidity, and NOx levels strongly affect air quality.
- The predictions can help monitor pollution trends and issue early warnings in unsafe air conditions.
- The small errors mean this model can be used for real-time environmental monitoring systems.

8. Conclusion

- Collected and analyzed the Air Quality Dataset.
- Cleaned and preprocessed the data (handled missing values, selected key features).
- Built a Linear Regression model to predict pollutant concentration.
- Achieved high accuracy ($R^2 = 0.95$), proving the model's reliability.
- Created visualizations like trend plots, predicted vs actual plots, and residuals for better understanding.

Challenges Faced

- Missing and inconsistent data required careful cleaning.
- Choosing the right features for prediction was time-consuming.
- Data imbalance between “safe” and “unsafe” values initially affected results.
- Understanding relationships between pollutants required detailed analysis.

Future Work

- Implement advanced regression models (Random Forest, SVR, Neural Networks) for better accuracy.
- Develop a real-time dashboard to display live air quality predictions.
- Include more environmental and meteorological features.

SVKM's NMIMS (Deemed-to-be-University)

- Develop a real-time air quality prediction system.
- Deploy the model as a web or mobile app for public air quality monitoring.
- Extend analysis to multi-level air quality categories (Good, Moderate, Poor, Hazardous).

10. References

- [Air Quality Dataset](#)
- Python libraries: Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn
- Tutorials, research papers, and **OpenAI ChatGPT** for coding guidance and explanations